
Bölüm-7

İSTİSNA YÖNETİMİ

OBJECTIVES



Bu bölümde;

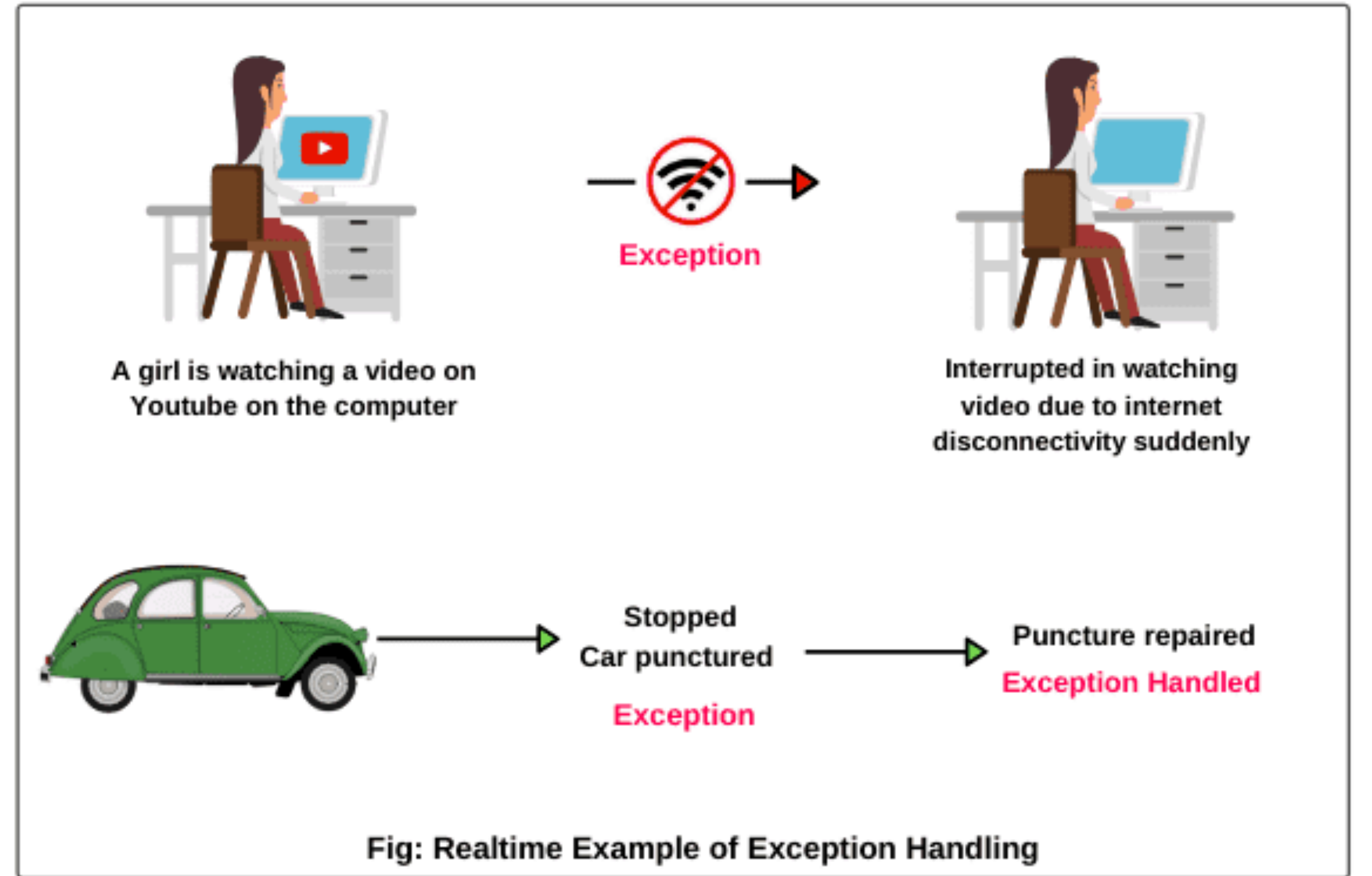
- ▶ İstisna işlemenin (yönetimi) çalışma zamanı problemlerin çözümünde nasıl etkin bir mekanizma sağladığını öğreneceksiniz.
- ▶ İstisnaların oluşabileceği kodu sınırlamak için “try” bloklarını kullanacaksınız.
- ▶ Bir istisna belirtmek için “throw” kullanacaksınız.
- ▶ İstisna işleyicileri belirtmek için “catch” bloklarını kullanacaksınız.
- ▶ İstisna işlemenin ne zaman kullanılacağını öğreneceksiniz.
- ▶ Exception sınıfı hiyerarşisini anlayacaksınız.
- ▶ Kaynakları serbest bırakmak için “finally” bloğu kullanacaksınız.
- ▶ Bir istisnayı yakalayıp diğerini atarak istisnaları zincirleyeceksiniz.
- ▶ Kullanıcı tanımlı istisnalar oluşturacaksınız.
- ▶ Bir yöntemde belirli bir noktada doğru olması gereken koşulları belirtmek için hata ayıklama özelliği “assert” kullanacaksınız.
- ▶ “try” bloğu sona erdiğinde bir kaynağı otomatik olarak nasıl serbest bırakabileceğini öğreneceksiniz.



- ▶ Giriş
- ▶ İstisna işleme örneği: “Sıfıra Bölme Hatası”
- ▶ İstisna işleme örneği: “ArithmeticExceptions”
- ▶ İstisna işleme örneği: “InputMismatchExceptions”
- ▶ İstisna işleme ne zaman kullanılır?
- ▶ Java’da EXCEPTION hiyerarşisi
- ▶ “finally” Blok kullanımı
- ▶ Bir istisnadan bilgi edinme
- ▶ “assert” kullanımı



- Bölüm 7-8'den bildiğiniz gibi, bir istisna, bir programın yürütülmesi sırasında bir sorun olduğunu gösterir.
- İstisna işleme, uygulamaların istisnaları çözmesini (veya işlemesini) sağlar. Bazı durumlarda, bir program hiçbir sorunla karşılaşılmamış gibi çalışmaya devam edebilir.
- Bu bölümde sunulan özellikler, sorunlarla başa çıkabilen ve sorunsuz bir şekilde yürütmeye devam edebilen veya sonlandırabilen sağlam ve hataya dayanıklı programlar yazmanıza yardımcı olur.



SOFTWARE ENGINEERING OBSERVATION



Endüstride, şirketlerin katı tasarım, kodlama, test etme, hata ayıklama ve bakım standartlarına sahip olduğunu göreceksiniz.

Şirketlerin genellikle gerçek zamanlı sistemler, yüksek performanslı matematiksel hesaplamalar, büyük veri, ağ tabanlı dağıtılmış sistemler vb. gibi uygulama türüne duyarlı kendi istisna işleme standartları vardır.

Chapter	Sample of exceptions used
Chapter 7	<code>ArrayIndexOutOfBoundsException</code>
Chapters 8–10	<code>IllegalArgumentException</code>
Chapter 11	<code>ArithmeticException</code> , <code>InputMismatchException</code>
Chapter 15	<code>SecurityException</code> , <code>FileNotFoundException</code> , <code>IOException</code> , <code>ClassNotFoundException</code> , <code>IllegalStateException</code> , <code>FormatterClosedException</code> , <code>NoSuchElementException</code>
Chapter 16	<code>ClassCastException</code> , <code>UnsupportedOperationException</code> , <code>NullPointerException</code> , custom exception types
Chapter 20	<code>ClassCastException</code> , custom exception types

Fig. 11.1 | Various exception types that you'll see throughout this book



- İlk olarak, istisna işleme kullanmayan bir uygulamada hatalar ortaya çıktığında ne olduğunu göstereceğiz.
- Şekil 11.2, kullanıcıdan iki tamsayı ister ve bunları tamsayı bölümünü hesaplayan ve bir int sonucu veren yöntem bölümüne iletir.
- Bu örnekte, bir yöntem bir sorun algıladığında ve onu çözemediğinde istisnaların atıldığını (yani istisna oluştuğunu) göreceksiniz.

```
1 // Fig. 11.2: DivideByZeroNoExceptionHandling.java
2 // Integer division without exception handling.
3 import java.util.Scanner;
4
5 public class DivideByZeroNoExceptionHandling {
6     // demonstrates throwing an exception when a divide-by-zero occurs
7     public static int quotient(int numerator, int denominator) {
8         return numerator / denominator; // possible division by zero
9     }
10
11     public static void main(String[] args) {
12         Scanner scanner = new Scanner(System.in);
13
14         System.out.print("Please enter an integer numerator: ");
15         int numerator = scanner.nextInt();
```

Fig. 11.2 | Integer division without exception handling. (Part 1 of 3.)



- İlk olarak, istisna işleme kullanmayan bir uygulamada hatalar ortaya çıktığında ne olduğunu göstereceğiz.
- Şekil 11.2, kullanıcıdan iki tamsayı ister ve bunları tamsayı bölümünü hesaplayan ve bir int sonucu veren yöntem bölümüne iletir.
- Bu örnekte, bir yöntem bir sorun algıladığında ve onu çözemediğinde istisnaların atıldığını (yani istisna oluştuğunu) göreceksiniz.

```
16      System.out.print("Please enter an integer denominator: ");
17      int denominator = scanner.nextInt();
18
19      int result = quotient(numerator, denominator);
20      System.out.printf(
21          "%nResult: %d / %d = %d%n", numerator, denominator, result);
22  }
23 }
```

```
Please enter an integer numerator: 100
Please enter an integer denominator: 7
```

```
Result: 100 / 7 = 14
```

Fig. 11.2 | Integer division without exception handling. (Part 2 of 3.)



- İlk olarak, istisna işleme kullanmayan bir uygulamada hatalar ortaya çıktığında ne olduğunu göstereceğiz.
- Şekil 11.2, kullanıcıdan iki tamsayı ister ve bunları tamsayı bölümünü hesaplayan ve bir int sonucu veren yöntem bölümüne iletir.
- Bu örnekte, bir yöntem bir sorun algıladığında ve onu çözemediğinde istisnaların atıldığını (yani istisna oluştuğunu) göreceksiniz.

```
Please enter an integer numerator: 100
Please enter an integer denominator: 0
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at DivideByZeroNoExceptionHandling.quotient(
        DivideByZeroNoExceptionHandling.java:8)
    at DivideByZeroNoExceptionHandling.main(
        DivideByZeroNoExceptionHandling.java:19)
```

```
Please enter an integer numerator: 100
Please enter an integer denominator: hello
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Unknown Source)
    at java.util.Scanner.next(Unknown Source)
    at java.util.Scanner.nextInt(Unknown Source)
    at java.util.Scanner.nextInt(Unknown Source)
    at DivideByZeroNoExceptionHandling.main(
        DivideByZeroNoExceptionHandling.java:17)
```

Fig. 11.2 | Integer division without exception handling. (Part 3 of 3.)



► Şimdi ise, istisnaları nasıl yöneteceğimizi görelim.

```
1 // Fig. 11.3: DivideByZeroWithExceptionHandling.java
2 // Handling ArithmeticExceptions and InputMismatchExceptions.
3 import java.util.InputMismatchException;
4 import java.util.Scanner;
5
6 public class DivideByZeroWithExceptionHandling
7 {
8     // demonstrates throwing an exception when a divide-by-zero occurs
9     public static int quotient(int numerator, int denominator)
10         throws ArithmeticException {
11         return numerator / denominator; // possible division by zero
12     }
13 }
```

Fig. 11.3 | Handling ArithmeticExceptions and InputMismatchExceptions. (Part 1 of 5.)



► Şimdi ise, istisnaları nasıl yöneteceğimizi görelim.

```
14 public static void main(String[] args) {
15     Scanner scanner = new Scanner(System.in);
16     boolean continueLoop = true; // determines if more input is needed
17
18     do {
19         try { // read two numbers and calculate quotient
20             System.out.print("Please enter an integer numerator: ");
21             int numerator = scanner.nextInt();
22             System.out.print("Please enter an integer denominator: ");
23             int denominator = scanner.nextInt();
24
25             int result = quotient(numerator, denominator);
26             System.out.printf("%nResult: %d / %d = %d%n", numerator,
27                             denominator, result);
28             continueLoop = false; // input successful; end looping
29         }
```

Fig. 11.3 | Handling ArithmeticExceptions and InputMismatchExceptions. (Part 2 of 5.)



► Şimdi ise, istisnaları nasıl yöneteceğimizi görelim.

```
30         catch (InputMismatchException inputMismatchException) {
31             System.err.printf("%nException: %s%n",
32                 inputMismatchException);
33             scanner.nextLine(); // discard input so user can try again
34             System.out.printf(
35                 "You must enter integers. Please try again.%n%n");
36         }
37         catch (ArithmeticException arithmeticException) {
38             System.err.printf("%nException: %s%n", arithmeticException);
39             System.out.printf(
40                 "Zero is an invalid denominator. Please try again.%n%n");
41         }
42     } while (continueLoop);
43 }
44 }
```

Fig. 11.3 | Handling ArithmeticExceptions and InputMismatchExceptions. (Part 3 of 5.)

Copyright © 2020 Pearson Education Ltd. All Rights Reserved



► Şimdi ise, istisnaları nasıl yöneteceğimizi görelim.

```
Please enter an integer numerator: 100
Please enter an integer denominator: 7

Result: 100 / 7 = 14
```

```
Please enter an integer numerator: 100
Please enter an integer denominator: 0

Exception: java.lang.ArithmeticException: / by zero
Zero is an invalid denominator. Please try again.

Please enter an integer numerator: 100
Please enter an integer denominator: 7

Result: 100 / 7 = 14
```

Fig. 11.3 | Handling ArithmeticExceptions and InputMismatchExceptions. (Part 4 of 5.)

Copyright © 2020 Pearson Education Ltd. All Rights Reserved



► Şimdi ise, istisnaları nasıl yöneteceğimizi görelim.

```
Please enter an integer numerator: 100
Please enter an integer denominator: hello

Exception: java.util.InputMismatchException
You must enter integers. Please try again.

Please enter an integer numerator: 100
Please enter an integer denominator: 7

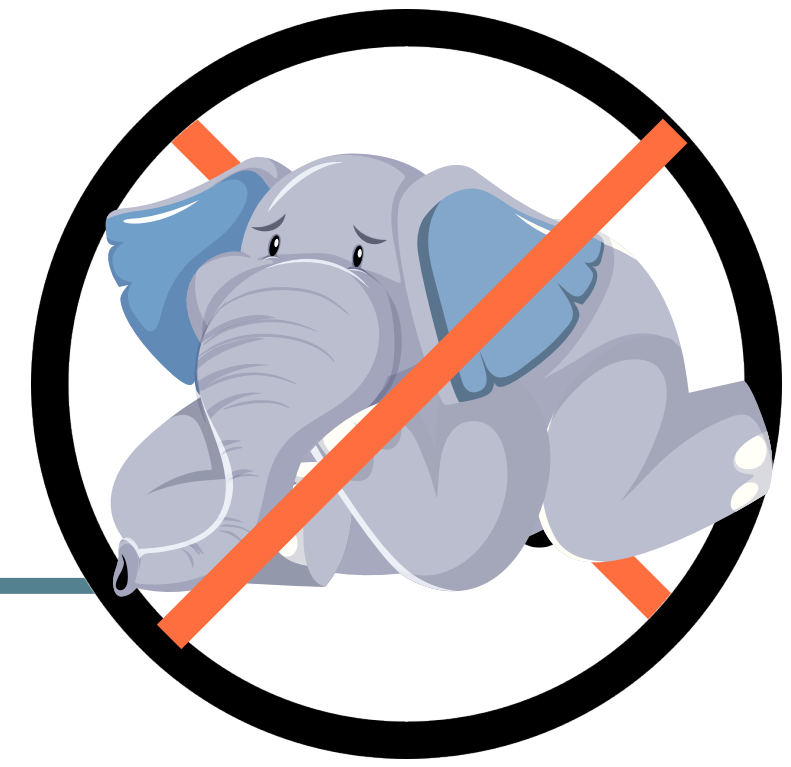
Result: 100 / 7 = 14
```

Fig. 11.3 | Handling ArithmeticExceptions and InputMismatchExceptions. (Part 5 of 5.)



- ▶ İstisna işleme, bir deyim yürütüldüğünde oluşan senkron hataları işlemek için tasarlanmıştır.
- ▶ Kitap boyunca göreceğimiz yaygın örnekler,
 - ▶ aralık dışı dizi indeksleri, (`ArrayIndexOutOfBoundsException`)
 - ▶ aritmetik taşma (`ArithmeticException`)
 - ▶ yani, temsil edilebilir değer aralığının dışındaki bir değer
 - ▶ sıfıra bölme, (`DivideByZeroException`)
 - ▶ geçersiz yöntem parametreleri (`IllegalArgumentException`)
 - ▶ iş parçacığı kesintisi. (`ThreadDeath`)

COMMON PROGRAMMING ERROR



Try ve catch bloklarının arasına kod eklerseniz, sözdizimi hatası alırsınız.

SOFTWARE ENGINEERING OBSERVATION



İstisna işleme ve hata kurtarma stratejinizi tasarım sürecinin başlangıcından itibaren sisteminize dahil edin.

Bir sistem uygulandıktan (kodladıktan) sonra bunları dahil etmek zor olabilir.

SOFTWARE ENGINEERING OBSERVATION



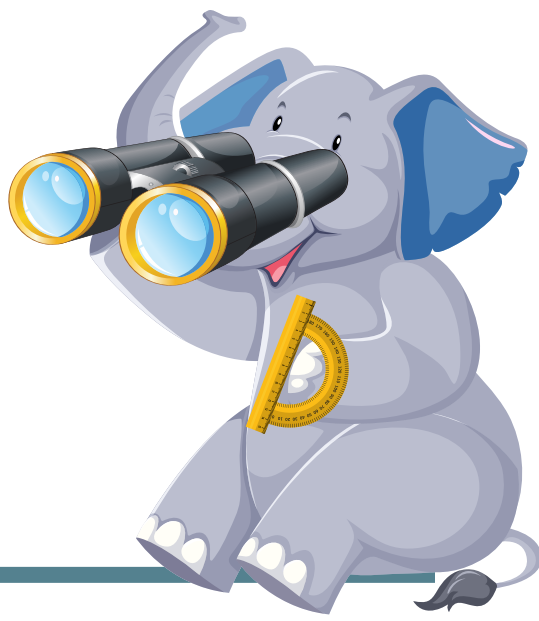
İstisna işleme, hataların belgelenmesi, saptanması ve düzeltilmesi için tek tip bir teknik sağlar. Bu, büyük projeler üzerinde çalışan programcıların birbirlerinin hata işleme kodunu anlamalarına yardımcı olur.

SOFTWARE ENGINEERING OBSERVATION



Çok çeşitli durumlar istisnalar oluşturabilir; bazı istisnalardan kurtulmak diğerlerinden daha kolaydır.

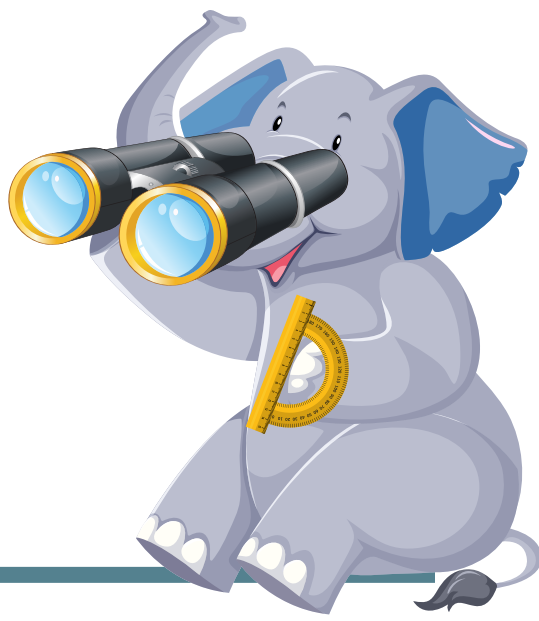
SOFTWARE ENGINEERING OBSERVATION



Bazen önce verileri doğrulayarak bir istisnayı önleyebilirsiniz.

Örneğin tamsayı bölme işlemini gerçekleştirmeden önce paydanın sıfır olmadığından emin olabilirsiniz, bu da sıfıra böldüğünüzde oluşan ArithmeticException'ı engeller.

SOFTWARE ENGINEERING OBSERVATION



Bazen önce verileri doğrulayarak bir istisnayı önleyebilirsiniz.

Örneğin tamsayı bölme işlemini gerçekleştirmeden önce paydanın sıfır olmadığından emin olabilirsiniz, bu da sıfıra böldüğünüzde oluşan `ArithmeticException`'ı engeller.

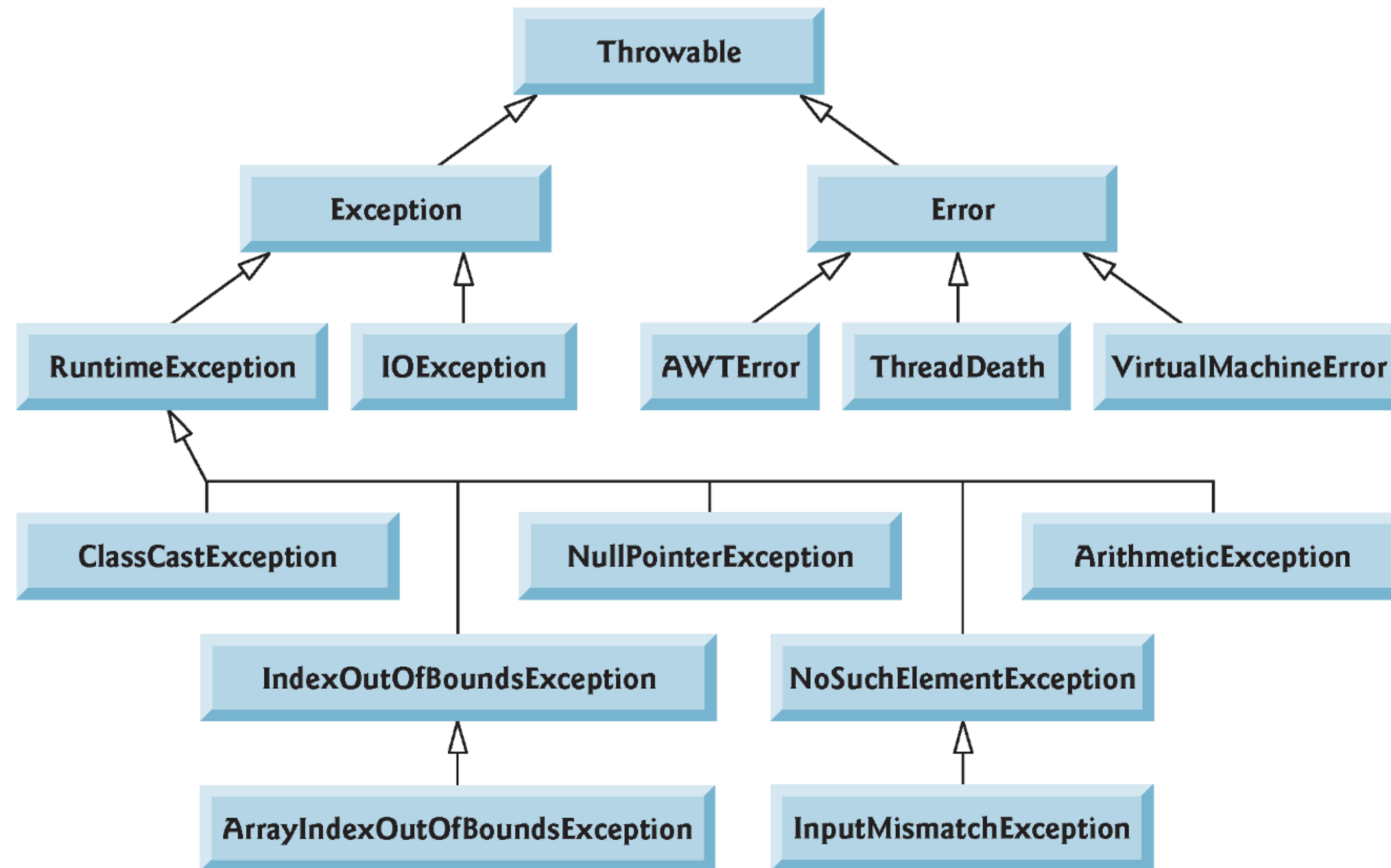


Fig. 11.4 | Portion of class `Throwable`'s inheritance hierarchy.



▶ Checked Exception:

▶ Derleyici tarafından kontrol edilir.

▶ `ClassNotFoundException`

▶ `IOException`

▶ `SQLException`

▶ Unchecked Exception:

▶ Çalışma zamanında ortaya çıkar.

▶ `ArithmeticException`

▶ `ArrayStoreException`

▶ `ClassCastException`



► İstisna oluşsa bile çalışacak kod bloktur.

```
1 // Fig. 11.5: UsingExceptions.java
2 // try...catch...finally exception handling mechanism.
3
4 public class UsingExceptions {
5     public static void main(String[] args) {
6         try {
7             throwException();
8         }
9         catch (Exception exception) { // exception thrown by throwException
10             System.err.println("Exception handled in main");
11         }
12
13         doesNotThrowException();
14     }
15 }
```

Fig. 11.5 | try...catch...finally exception-handling mechanism. (Part 1 of 4.)



► İstisna oluşsa bile çalışacak kod bloktur.

```
16 // demonstrate try...catch...finally
17 public static void throwException() throws Exception {
18     try { // throw an exception and immediately catch it
19         System.out.println("Method throwException");
20         throw new Exception(); // generate exception
21     }
22     catch (Exception exception) { // catch exception thrown in try
23         System.err.println(
24             "Exception handled in method throwException");
25         throw exception; // rethrow for further processing
26
27         // code here would not be reached; would cause compilation errors
28     }
29
30     finally { // executes regardless of what occurs in try...catch
31         System.err.println("Finally executed in throwException");
32     }
33
34     // code here would not be reached; would cause compilation errors
35 }
```

Fig. 11.5 | try...catch...finally exception-handling mechanism. (Part 2 of 4.)



► İstisna oluşsa bile çalışacak kod bloktur.

```
36
37 // demonstrate finally when no exception occurs
38 public static void doesNotThrowException() {
39     try { // try block does not throw an exception
40         System.out.println("Method doesNotThrowException");
41     }
42     catch (Exception exception) { // does not execute
43         System.err.println(exception);
44     }
45     finally { // executes regardless of what occurs in try...catch
46         System.err.println("Finally executed in doesNotThrowException");
47     }
48
49     System.out.println("End of method doesNotThrowException");
50 }
51 }
```

Fig. 11.5 | try...catch...finally exception-handling mechanism. (Part 3 of 4.)



► İstisna oluşsa bile çalışacak kod bloktur.

```
Method throwException  
Exception handled in method throwException  
Finally executed in throwException  
Exception handled in main  
Method doesNotThrowException  
Finally executed in doesNotThrowException  
End of method doesNotThrowException
```

Fig. 11.5 | try...catch...finally exception-handling mechanism. (Part 4 of 4.)



► İstisna oluştuğunda, yığın onu bulana kadar yukarı doğru çalışır. (Sınıftan-metoda-satıra)

```
1 // Fig. 11.6: UsingExceptions.java
2 // Stack unwinding and obtaining data from an exception object.
3
4 public class UsingExceptions {
5     public static void main(String[] args) {
6         try {
7             method1();
8         }
9         catch (Exception exception) { // catch exception thrown in method1
10            System.err.printf("%s%n%n", exception.getMessage());
11            exception.printStackTrace();
12
13            // obtain the stack-trace information
14            StackTraceElement[] traceElements = exception.getStackTrace();
15        }
```

Fig. 11.6 | Stack unwinding and obtaining data from an exception object. (Part 1 of 4.)



► İstisna oluştuğunda, yığın onu bulana kadar yukarı doğru çalışır. (Sınıftan-metoda-satıra)

```
16      System.out.printf("%nStack trace from getStackTrace:%n");
17      System.out.println("Class\t\tFile\t\t\tLine\tMethod");
18
19      // loop through traceElements to get exception description
20      for (StackTraceElement element : traceElements) {
21          System.out.printf("%s\t", element.getClassName());
22          System.out.printf("%s\t", element.getFileName());
23          System.out.printf("%s\t", element.getLineNumber());
24          System.out.printf("%s%n", element.getMethodName());
25      }
26  }
27 }
28
```

Fig. 11.6 | Stack unwinding and obtaining data from an exception object. (Part 2 of 4.)



► İstisna oluştuğunda, yığın onu bulana kadar yukarı doğru çalışır. (Sınıftan-metoda-satıra)

```
29 // call method2; throw exceptions back to main
30 public static void method1() throws Exception {
31     method2();
32 }
33
34 // call method3; throw exceptions back to method1
35 public static void method2() throws Exception {
36     method3();
37 }
38
39 // throw Exception back to method2
40 public static void method3() throws Exception {
41     throw new Exception("Exception thrown in method3");
42 }
43 }
```

Fig. 11.6 | Stack unwinding and obtaining data from an exception object. (Part 3 of 4.)



► İstisna oluştuğunda, yığın onu bulana kadar yukarı doğru çalışır. (Sınıftan-metoda-satıra)

Exception thrown in method3

```
java.lang.Exception: Exception thrown in method3
    at UsingExceptions.method3(UsingExceptions.java:41)
    at UsingExceptions.method2(UsingExceptions.java:36)
    at UsingExceptions.method1(UsingExceptions.java:31)
    at UsingExceptions.main(UsingExceptions.java:7)
```

Stack trace from getStackTrace:

Class	File	Line	Method
UsingExceptions	UsingExceptions.java	41	method3
UsingExceptions	UsingExceptions.java	36	method2
UsingExceptions	UsingExceptions.java	31	method1
UsingExceptions	UsingExceptions.java	7	main

Fig. 11.6 | Stack unwinding and obtaining data from an exception object. (Part 4 of 4.)



- ▶ Assert, programda yapılmış herhangi bir varsayımın doğruluğunu test etmeye izin verir.
- ▶ Java “assert” yürütürken, bunun doğru olduğuna varsayılır, başarısız olursa, JVM AssertionError adlı bir hata atar.

```

1  // Fig. 11.8: AssertTest.java
2  // Checking with assert that a value is within range
3  import java.util.Scanner;
4
5  public class AssertTest {
6      public static void main(String[] args) {
7          Scanner input = new Scanner(System.in);
8
9          System.out.print("Enter a number between 0 and 10: ");
10         int number = input.nextInt();
11
12         // assert that the value is >= 0 and <= 10
13         assert (number >= 0 && number <= 10) : "bad number: " + number;
14
15         System.out.printf("You entered %d\n", number);
16     }
17 }

```

Fig. 11.8 | Checking with assert that a value is within range. (Part 1 of 2.)



- ▶ Assert, programda yapılmış herhangi bir varsayımın doğruluğunu test etmeye izin verir.
- ▶ Java "assert" yürütürken, bunun doğru olduğuna varsayılır, başarısız olursa, JVM AssertionError adlı bir hata atar.

```
Enter a number between 0 and 10: 5  
You entered 5
```

```
Enter a number between 0 and 10: 50  
Exception in thread "main" java.lang.AssertionError: bad number: 50  
    at AssertTest.main(AssertTest.java:13)
```

Fig. 11.8 | Checking with assert that a value is within range. (Part 2 of 2.)